API SECURITY REPORT

Intelligent API Security Testing & Monitoring Platform

Target: http://localhost:5001

Session ID: #1

Date: 2026-08-20

10.0/10

Risk Level: CRITICAL

Executive Summary

Target URL:	http://localhost:5001
Total Endpoints Discovered:	12
Total Vulnerabilities:	16
Auth Test Results:	3
Critical Findings:	5
High Findings:	10
Medium Findings:	3
Low Findings:	1
Overall Risk Score:	10.0/10
Risk Level:	CRITICAL

AI Risk Assessment Summary

Security Score: 0.0/100 | Risk Score: 10.0/10 (CRITICAL). Found 19 vulnerabilities: 5 Critical, 10 High. ⚠ Immediate remediation required before any production deployment.

Vulnerability Findings

Vulnerability Type	Severity	CVSS	OWASP	Path
Mass Assignment	HIGH	7.5	API3	/
Cross-Site Scripting (XSS)	HIGH	7.2	API8	/
Broken Authentication	CRITICAL	9.1	API2	/
Broken Authentication (alg:none tok	CRITICAL	9.8	API2	/
Broken Authentication	CRITICAL	9.1	API2	/
Broken Authentication (alg:none tok	CRITICAL	9.8	API2	/
Default Credentials Accepted	CRITICAL	9.8	API2	/
Broken Object Level Authorization (	HIGH	8.1	API1	/
Broken Object Level Authorization (	HIGH	8.1	API1	/
Broken Object Level Authorization (	HIGH	8.1	API1	/
Broken Function Level Authorization	HIGH	8.0	API5	/
Excessive Data Exposure	HIGH	7.5	API3	/
Excessive Data Exposure	HIGH	7.5	API3	/
Excessive Data Exposure	HIGH	7.5	API3	/
Information Disclosure via Health E	LOW	3.1	API9	/
Missing Rate Limiting	MEDIUM	5.3	API4	/

Remediation Recommendations

1. Fix Broken Authentication

- Enforce JWT signature validation — never accept alg:none tokens
- Implement token expiry (short-lived access tokens, long-lived refresh tokens)
- Add account lockout after N failed login attempts
- Use multi-factor authentication (MFA) for sensitive operations
- Rotate API keys and tokens regularly; invalidate on logout
- Log all authentication events and alert on anomalies

2. Remove Default Credentials

- Remove all default usernames and passwords before deployment
- Force password change on first login for any seeded accounts
- Implement a strong password policy (length, complexity, history)
- Use a secrets manager (e.g., HashiCorp Vault) for all credentials

3. Fix Broken Object Level Authorization (BOLA/IDOR)

- Validate that the authenticated user owns the requested resource on every request
- Never rely solely on object IDs in URLs — verify ownership server-side
- Use non-sequential, unpredictable IDs (e.g., UUIDs) to reduce guessability
- Implement row-level security in the database layer
- Write automated tests that verify cross-user access is denied

4. Fix Broken Function Level Authorization (BFLA)

- Enforce role-based access control (RBAC) on every administrative endpoint
- Check the caller's role on every request, not just at login time
- Deny by default — require explicit permission grants for all sensitive functions
- Audit all admin-level functions to ensure non-admin accounts cannot reach them
- Use middleware or decorators to enforce function-level authorization consistently

5. Reduce Excessive Data Exposure

- Apply response field filtering — only return fields the client actually needs
- Define explicit response DTOs/serializers with allowlists, not blocklists
- Never expose password hashes, API keys, SSNs, or financial data in API responses
- Use field-level encryption for sensitive attributes stored in the database
- Audit all API responses and remove unnecessary sensitive fields

6. Prevent Mass Assignment

- Use explicit allowlist serializers — never auto-bind all request fields
- Define separate DTOs/schemas for create, update, and response
- Mark sensitive fields (is_admin, role, balance) as read-only in your ORM
- Validate the shape and types of all incoming data strictly

7. Prevent Cross-Site Scripting

- HTML-encode all user-supplied output before rendering
- Use a Content Security Policy (CSP) header to restrict script sources
- Sanitize input with a server-side library (e.g., bleach for Python)
- Set HttpOnly and Secure flags on all cookies
- Use the X-XSS-Protection header as a browser-level defense

8. Implement Rate Limiting

- Apply rate limits per IP and per authenticated user on all endpoints
- Use Flask-Limiter or API Gateway rate limiting rules
- Return proper 429 Too Many Requests responses with Retry-After headers
- Implement exponential back-off on repeated failures
- Monitor for unusual traffic spikes and set alerts

9. Reduce Information Disclosure

- Suppress detailed error messages in production (stack traces, SQL errors)
- Remove or obscure Server, X-Powered-By, X-AspNet-Version headers
- Avoid exposing internal paths, library versions, or infrastructure details
- Return generic error messages to clients; log full details server-side

10. Implement API Authentication Centrally

Use a single auth middleware or gateway to enforce authentication uniformly across all routes.